

InsurancelQ

AI-Powered Insurance Intelligence Platform

Full Technical Project Documentation

React 19 Frontend | Spring Boot 3.2.5 Core API
Python FastAPI ML Service | Node.js Socket.IO Notifications
AWS EKS Kubernetes | Jenkins CI/CD | Prometheus Monitoring

Repository: github.com/Neel-Asher/ClaimsManagementApp

Branch: dev | Version: 1.0.0

Table of Contents

Table of Contents	2
1. Project Overview	5
1.1 Core Capabilities	5
1.2 Repository Branch Structure	5
2. System Architecture	7
2.1 Architecture Diagram	7
2.2 Inter-Service Communication	7
2.3 Request Flow Walkthrough	8
3. Complete Technology Stack	9
3.1 Application Layer	9
3.2 Infrastructure & Cloud Layer	10
3.3 Testing Frameworks	10
4. Service 1 — React Frontend	12
4.1 Pages and Role Access	12
4.2 Folder Structure	12
4.3 Authentication Flow	13
4.4 Environment Variables	13
4.5 Quick Start	13
5. Service 2 — Spring Boot Core API	14
5.1 Package Structure	14
5.2 REST API Reference	14
Authentication	14
Claims	14
Policies and Customers	15
Fraud and Analytics	15
5.3 Security Configuration	15
5.4 Key Configuration (application.properties)	16
5.5 Quick Start	16
6. Service 3 — Python ML & Intelligence Service	17
6.1 Fraud Detection Modes	17
Rule-Based Mode (FRAUD_DETECTION_MODE=RULE_BASED)	17
ML Mode (FRAUD_DETECTION_MODE=ML)	17
6.2 ML Feature Vector	17
6.3 Key Endpoints	17
6.4 ETL Pipeline	18

6.5 Folder Structure.....	18
6.6 Quick Start	19
7. Service 4 — Node.js Notification Service	20
7.1 WebSocket Architecture	20
Connection Setup	20
Notification Payload Shape	20
7.2 Cron Jobs	20
7.3 REST Endpoints	21
7.4 Database Schema	21
7.5 Quick Start	21
8. Service 5 — Cloud & DevOps Deployment	23
8.1 Dockerfiles	23
8.2 Kubernetes Resource Map	23
8.3 NGINX Ingress Routing	24
8.4 CI/CD Pipeline (Jenkins)	24
8.5 Kubernetes Secrets & ConfigMap.....	24
Secret: insuranceiq-secrets.....	24
ConfigMap: insuranceiq-configmap.....	25
8.6 Monitoring — Prometheus & Grafana	25
8.7 Deployment Walkthrough	25
9. Data Model & Database Schema	27
9.1 Core Database — insuranceiq_db.....	27
9.2 Notification Database — insuranceiq_notifications	27
9.3 ML Database — insuranceiq.db (SQLite) or MySQL (Prod).....	28
10. Authentication & Security	29
10.1 JWT Configuration.....	29
10.2 Roles & Permissions	29
11. Local Development Setup	30
11.1 Prerequisites	30
11.2 Setup Steps.....	30
Step 1 — Create MySQL Databases	30
Step 2 — Start Spring Boot (Terminal 1).....	30
Step 3 — Start Python ML Service (Terminal 2).....	30
Step 4 — Start Node.js Notification Service (Terminal 3)	30
Step 5 — Start React Frontend (Terminal 4).....	31
11.3 Service Health Check URLs	31

- 12. Environment Variables Reference 32
 - 12.1 React Frontend (client/.env) 32
 - 12.2 Spring Boot (application.properties / env) 32
 - 12.3 Python ML Service (.env) 32
 - 12.4 Node.js Notification Service (notification-service/.env)..... 32
- Appendix: AWS Services Reference 34

1. Project Overview

InsuranceIQ is an AI-powered, cloud-native enterprise platform that automates the full insurance lifecycle — from policy issuance and customer onboarding through claims processing, fraud detection, and real-time notifications. It is built as a distributed microservices system using four independently deployable application services and a dedicated cloud/DevOps layer hosted on AWS EKS.

The platform enables insurance companies to manage policy issuance, customer KYC, agent performance, and end-to-end claims processing. A machine learning service automatically scores claims for fraud risk before settlement. A dedicated Node.js service delivers real-time push notifications to all stakeholders over WebSocket connections.

1.1 Core Capabilities

- Role-based multi-dashboard SPA for Admin, Agent, Customer, and Claims Manager
- Full claims lifecycle: submission → review → ML fraud scoring → approval/rejection → settlement
- ML-powered fraud detection with switchable rule-based and scikit-learn Random Forest modes
- Real-time push notifications via Socket.IO for claim events, KYC updates, and policy renewals
- Automated daily policy renewal reminders via node-cron scheduled jobs
- Analytics dashboards with live Recharts visualizations across all business dimensions
- JWT-based authentication with role-level access control shared across all services
- AWS EKS deployment with Jenkins CI/CD, ECR image registry, and Prometheus/Grafana monitoring

1.2 Repository Branch Structure

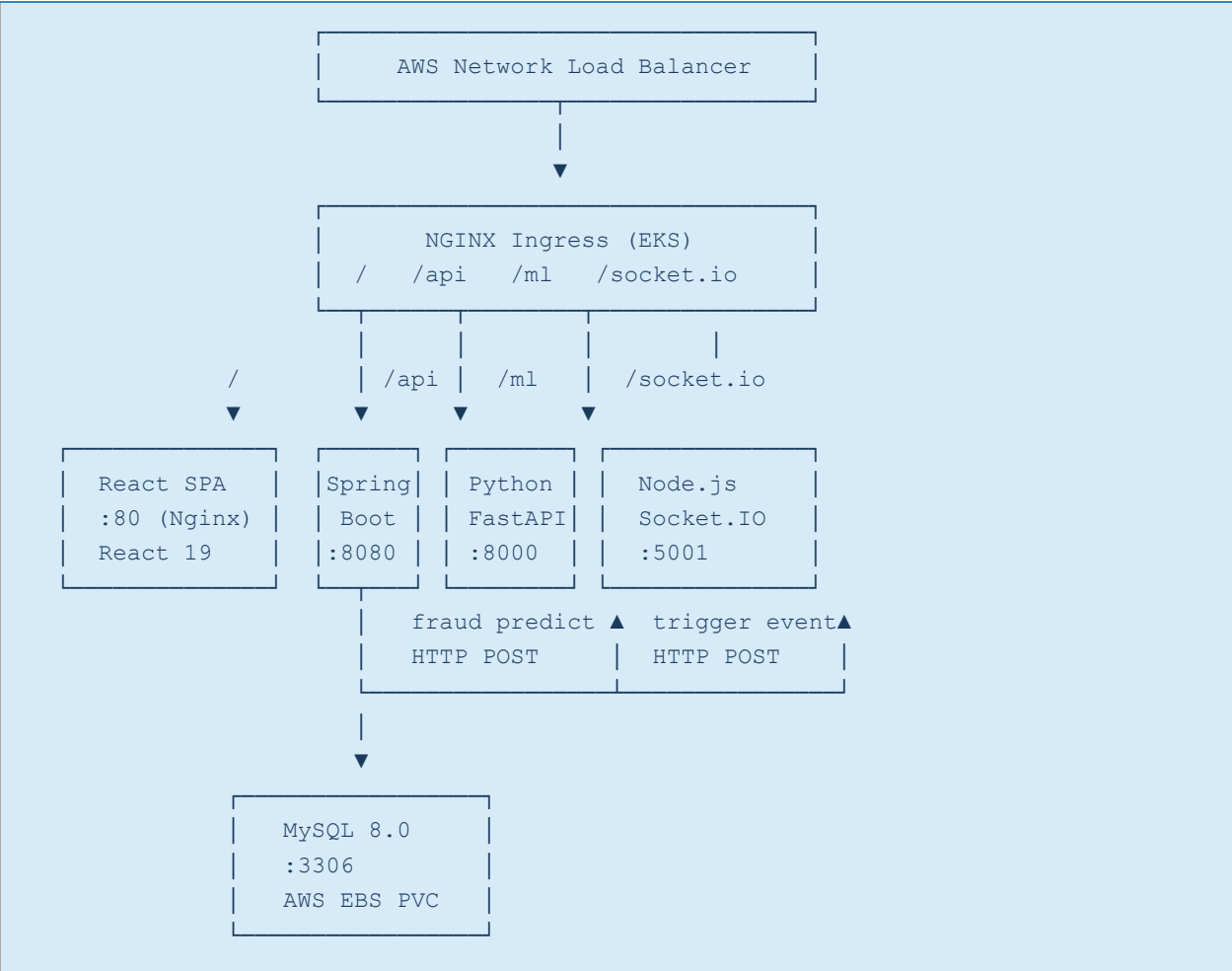
Branch	Contents	Service README
main	Platform overview and architecture diagram	Root README.md
dev	Integration branch — all services combined	README_dev.md (this doc)
dev-react-frontend	React 19 + Vite 8 SPA	README_dev-react-frontend.md
dev-spring-backend	Spring Boot 3.2.5 Core API + MySQL	README_dev-spring-backend.md
dev-python-ml-service	FastAPI + scikit-learn Fraud Detection + ETL	README_dev-python-ml-service.md

dev-node-notifications-backend	Node.js + Socket.IO + MySQL Notification Service	README_dev-node-notifications-backend.md
dev-cloud-deployment	Docker + EKS + Jenkins CI/CD + Prometheus/Grafana	README_dev-cloud-deployment.md

2. System Architecture

InsurancelQ follows a microservices architecture with clear separation of concerns. The React frontend communicates exclusively with the Spring Boot API. Spring Boot acts as the orchestration hub, calling both the Python ML service and the Node.js notification service. The Node.js service also has a scheduled cron job that queries Spring Boot for policy data.

2.1 Architecture Diagram



2.2 Inter-Service Communication

Caller	Target	Protocol	Trigger / Purpose
React Frontend	Spring Boot	HTTP REST	All user actions: auth, claims, policies
React Frontend	Node.js Notification Svc	WebSocket (Socket.IO)	Post-login connect for real-time push

Spring Boot	Python ML Service	HTTP POST	Fraud scoring on claim submission
Spring Boot	Node.js Notification Svc	HTTP POST	Claim update, KYC completion, renewal alert
Node.js Cron Job	Spring Boot	HTTP GET	Daily policy expiry scan for reminders

2.3 Request Flow Walkthrough

A full end-to-end claim submission flows through the system as follows:

- User submits a claim on the React SPA — React calls POST /api/claims on Spring Boot
- Spring Boot persists the claim to MySQL and immediately calls POST /predict/fraud/{id} on the Python ML service
- Python ML service runs fraud scoring (rule-based or Random Forest), returns probability and risk status
- Spring Boot stores the fraud prediction and calls POST /events/trigger on the Node.js notification service
- Node.js emits a real-time 'notification' event via Socket.IO to the Claims Manager's browser room
- React NotificationsCenter component receives the event and updates the unread badge in real time

3. Complete Technology Stack

3.1 Application Layer

Service	Technology	Version	Role
Frontend	React	^19.2.6	UI framework
Frontend	Vite	^8.0.12	Build tool and dev server
Frontend	React Router DOM	^7.15.1	Client-side routing with role guards
Frontend	Tailwind CSS	^4.3.0	Utility-first CSS styling
Frontend	Axios	^1.16.1	HTTP client with JWT interceptors
Frontend	Socket.IO Client	^4.8.3	WebSocket client for real-time events
Frontend	Recharts	^3.8.1	Chart library for analytics dashboards
Backend API	Spring Boot	3.2.5	Application framework
Backend API	Java	17	Runtime language
Backend API	Spring Security	via Boot 3	JWT filter chain, RBAC
Backend API	Spring Data JPA / Hibernate	6.x	ORM and DB persistence
Backend API	JJWT	0.12.5	JWT generation and validation (HS256)
Backend API	Springdoc OpenAPI	2.3.0	Auto-generated Swagger UI docs
Backend API	Lombok	Latest	Boilerplate reduction (getters, builders)
ML Service	Python	3.11+	Runtime
ML Service	FastAPI	>=0.110.0	Async REST framework
ML Service	Uvicorn	>=0.29.0	ASGI server
ML Service	scikit-learn	>=1.4.0	Random Forest fraud classifier
ML Service	pandas / numpy	>=2.2.0 / 1.26.0	Data processing and feature engineering
ML Service	SQLAlchemy	>=2.0.0	ORM for SQLite (dev) / MySQL (prod)
ML Service	Pydantic + Settings	>=2.6.0	Request validation and config management
Notification Svc	Node.js	>=18.x	JavaScript runtime
Notification Svc	Express	^4.19.2	HTTP server and REST routing
Notification Svc	Socket.IO	^4.7.5	WebSocket with room support

Notification Svc	mysql2	^3.10.0	MySQL driver (promise-based)
Notification Svc	jsonwebtoken	^9.0.2	JWT validation for socket auth
Notification Svc	node-cron	^3.0.3	Scheduled policy renewal reminders

3.2 Infrastructure & Cloud Layer

Layer	Technology	Version / Notes
Primary Database	MySQL Community Server	8.0
ML Database (dev)	SQLite (via aiosqlite)	In-process, zero config
ML Database (prod)	MySQL / PostgreSQL (via pymysql)	Set DATABASE_URL env var
Containers	Docker	Latest
Orchestration	AWS EKS (Kubernetes)	EKS v1.34
EKS Provisioning	eksctl	Latest
Image Registry	AWS ECR	Per-service repositories
Ingress	NGINX Ingress Controller	Installed via Helm
Load Balancer	AWS Network Load Balancer (NLB)	Auto-provisioned by EKS
Persistent Storage	AWS EBS CSI Driver	gp2 StorageClass, 10Gi PVC
CI/CD	Jenkins (Groovy Jenkinsfile)	Pipeline as Code
Monitoring	Prometheus + Grafana	kube-prometheus-stack Helm chart
Maven	Apache Maven	3.8+ (Spring Boot build)
Node Package Manager	npm	9.x+
Python Package Manager	pip / venv	Python 3.11+

3.3 Testing Frameworks

Service	Framework	Command	Coverage Focus
Spring Boot	JUnit 5 + Mockito	mvn test	ClaimService CRUD, exception handling, notification firing
Python ML	Pytest + HTTPX TestClient	pytest	Health endpoints, fraud inference, analytics queries
Node.js	Jest + Supertest	npm test	Health check, JWT 401 rejection, event endpoint, 404 handling

React	Vitest + React Testing Library	npm test	FraudReport render, stat cards, loading and empty states
-------	--------------------------------	----------	--

4. Service 1 — React Frontend

React 19 + Vite
8

Branch: dev-react-frontend | Port: 5173 (dev) / 80 (prod) | Node.js 18+

The React frontend is a single-page application that provides role-aware dashboards and workflows for four user types. It communicates with the Spring Boot backend via REST and with the Node.js notification service via WebSocket using Socket.IO.

4.1 Pages and Role Access

Route	Page / Component	Allowed Roles
/login	LoginPage	Public
/admin	AdminDashboard	admin
/agent	AgentDashboard	agent
/customer	CustomerDashboard	customer
/customers	CustomerManagement	admin, agent
/policies	PolicyManagement	admin, agent
/claims	ClaimsSubmission	All authenticated roles
/claims-workflow	ClaimsWorkflow	admin, claims_manager
/fraud-report	FraudReport	admin, claims_manager
/analytics	AnalyticsDashboard	admin, claims_manager
/notifications	NotificationsCenter	All authenticated roles
* (catch-all)	Redirect to /login	—

4.2 Folder Structure

```
client/
  Dockerfile           # Multi-stage: Node 20 Alpine → Nginx Alpine
  nginx.conf           # SPA routing: try_files $uri /index.html
  vite.config.js       # Vite + React + Tailwind plugins
  src/
    main.jsx           # ReactDOM.createRoot + BrowserRouter
    App.jsx            # Root routes + ProtectedRoute guards
    context/
      AuthContext.jsx  # Global JWT, user, and role state
    services/
      api.js           # Axios instance: JWT interceptor + 401 auto-logout
```

```
socketService.js      # Socket.IO: connect, rooms, auto-reconnect (5x)
authService.js        # Login, logout, token management
dataService.js        # API call functions per business domain
components/
  layout/              # DashboardLayout, Sidebar, TopNavbar, ProtectedRoute
  common/              # DataTable, StatCard, StatusBadge
  charts/Charts.jsx    # Recharts wrappers (line, bar, pie)
  modals/Modal.jsx     # Generic modal dialog wrapper
pages/                # One subfolder per domain
  admin / agent / customer / auth
  claims / customerMgmt / policy
  fraud / analytics / notifications
hooks/useApiData.js    # Shared loading/error/data pattern
utils/helpers.js       # Date, currency, status formatters
```

4.3 Authentication Flow

- User submits credentials on /login
- authService calls POST /api/auth/login — receives { token, user, role }
- Token is stored in localStorage; AuthContext is updated with user info
- ProtectedRoute checks AuthContext.user and role before rendering any page
- Axios interceptor attaches Authorization: Bearer <token> on every request
- On HTTP 401 response: interceptor clears localStorage and redirects to /login
- socketService.connect(userId) is called post-login for real-time event delivery

4.4 Environment Variables

Variable	Value / Description
VITE_SPRING_API_URL	http://localhost:8080/api — Spring Boot REST base URL
VITE_SOCKET_URL	http://localhost:3001 — Node.js WebSocket server URL

4.5 Quick Start

```
git checkout dev-react-frontend
cd client && npm install
cp .env.example .env      # set VITE_SPRING_API_URL and VITE_SOCKET_URL
npm run dev                # http://localhost:5173
npm run build              # production output to client/dist/
```

5. Service 2 — Spring Boot Core API

Spring Boot 3.2.5 / Java 17

Branch: dev-spring-backend | Port: 8080 | MySQL 8.0

The Spring Boot backend is the central hub of the InsuranceIQ platform. It exposes a RESTful API consumed by the React frontend, manages all business entities, handles JWT authentication with role-based access control, and orchestrates calls to the Python ML and Node.js notification services.

5.1 Package Structure

com.insuranceiq/	
controller/	REST controllers (Auth, Agent, Claim, Customer, Policy, Product, Fraud, Analytics, FileUpload)
service/	Business logic (one per domain) + FraudService (calls Python ML) + NotificationService (calls Node.js)
model/	JPA entities: User, Agent, Customer, Policy, Claim, ClaimDocument, Payment, FraudPrediction, InsuranceProduct
model/enums/	ClaimStatus, KycStatus, PaymentStatus, PaymentType, PolicyStatus, ProductType, Role
dto/	Request/Response objects per endpoint
repository/	Spring Data JPA interfaces (one per entity)
security/	JwtUtil, JwtFilter, SecurityConfig, UserDetailsServiceImpl
exception/	GlobalExceptionHandler, ResourceNotFoundException, BadRequestException, UnauthorizedException
util/	DataSeeder (seeds initial data on startup)

5.2 REST API Reference

Authentication

Endpoint	Description
POST /api/auth/login	Authenticate user — returns JWT token, user info, and role
POST /api/auth/register	Register a new user account

Claims

Endpoint	Description
----------	-------------

GET /api/claims	List all claims (filtered by role)
POST /api/claims	File a new insurance claim
GET /api/claims/{id}	Retrieve claim details by ID
PUT /api/claims/{id}/status	Update claim status (Admin / Claims Manager only)
POST /api/files/upload/{claimId}	Upload supporting document for a claim

Policies and Customers

Endpoint	Description
GET /api/policies	List all policies
POST /api/policies	Create a new policy (Admin / Agent)
GET /api/policies/{id}	Get policy details
PUT /api/policies/{id}	Update policy (Admin / Agent)
GET /api/customers	List all customers (Admin / Agent)
POST /api/customers	Create customer profile
PUT /api/customers/{id}	Update customer / KYC status

Fraud and Analytics

Endpoint	Description
POST /api/fraud/predict/{claimId}	Trigger ML fraud prediction for a claim
GET /api/fraud/report	Retrieve all fraud prediction results
GET /api/analytics/dashboard	Aggregated KPI summary for dashboards
GET /api/analytics/claims	Claims trend data for charts

5.3 Security Configuration

- Algorithm: HS256 — secret injected via APP_JWT_SECRET environment variable
- Token expiry: 24 hours (86,400,000 ms) — configurable via app.jwt.expiration-ms
- JwtFilter extends OncePerRequestFilter — extracts and validates Bearer token on every request
- SecurityConfig: stateless session (SessionCreationPolicy.STATELESS), CORS configured
- Role enforcement via @PreAuthorize annotations and HttpSecurity matchers
- UserDetailsServiceImpl loads users by email from MySQL for Spring Security

5.4 Key Configuration (application.properties)

```
server.port=8080

# MySQL
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/insuranceiq_db...}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:root}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:root}

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

# JWT
app.jwt.secret=${APP_JWT_SECRET:InsuranceIQSecretKey2024...}
app.jwt.expiration-ms=86400000

# External Services
app.fraud-service.url=${APP_FRAUD_SERVICE_URL:http://localhost:8000}
app.notification-service.url=${APP_NOTIFICATION_SERVICE_URL:http://localhost:5001}

# Swagger
springdoc.swagger-ui.path=/swagger-ui.html
```

5.5 Quick Start

```
git checkout dev-spring-backend
# Create MySQL database: CREATE DATABASE insuranceiq_db;
cd server
# Update application.properties with your credentials
mvn spring-boot:run
# API: http://localhost:8080
# Swagger UI: http://localhost:8080/swagger-ui.html
```

6. Service 3 — Python ML & Intelligence Service

Python 3.11 +
FastAPI + scikit-
learn

Branch: dev-python-ml-service | Port: 8000 | SQLite (dev) / MySQL (prod)

The Python ML service is an independent microservice called exclusively by Spring Boot — never directly by the React frontend. It provides three capabilities: ML-powered fraud detection, CSV/ETL data ingestion, and analytics aggregation. It is built with FastAPI for high performance and auto-generated OpenAPI documentation.

6.1 Fraud Detection Modes

Rule-Based Mode (FRAUD_DETECTION_MODE=RULE_BASED)

Applies weighted heuristics based on claim amount relative to policy coverage, time elapsed since policy inception (early claims are higher risk), number of prior claims by the same customer, and surveyor/assessor mismatch flags. No model training required.

ML Mode (FRAUD_DETECTION_MODE=ML)

Uses a trained scikit-learn Random Forest classifier. Before enabling, run: `python ml/train_model.py` to produce `ml/fraud_model.pkl`.

6.2 ML Feature Vector

Feature Name	Description
claim_amount	The total claimed value in currency units
days_since_policy_start	Number of days between policy start date and claim filing
claim_type	Encoded: motor=0, health=1, property=2, life=3
previous_claims_count	Number of prior claims filed by the same customer
customer_age	Customer age in years at time of claim
policy_premium_ratio	claim_amount divided by annual premium
surveyor_mismatch_flag	1 if the assigned surveyor is anomalous, 0 otherwise

6.3 Key Endpoints

Method + Endpoint	Auth	Description
GET /	Public	Health check — returns service status and version

POST /predict/fraud/{claim_id}	Via Spring Boot	Run fraud prediction for a specific claim ID
POST /predict/fraud/batch	Via Spring Boot	Batch fraud prediction for multiple claims
GET /predict/fraud/report	Via Spring Boot	Retrieve all stored fraud predictions
POST /etl/upload	Via Spring Boot	Upload a CSV file for ETL ingestion
POST /etl/process	Via Spring Boot	Process the most recently uploaded CSV
GET /etl/status	Via Spring Boot	Check status of the last ETL run
GET /analytics/summary	Via Spring Boot	Platform-level KPI aggregates
GET /analytics/claims/trend	Via Spring Boot	Claims volume over a time window
GET /analytics/fraud/distribution	Via Spring Boot	Fraud risk level breakdown (Low/Med/High)

6.4 ETL Pipeline

The ETL service ingests CSV files from the /data/sample/ directory or via HTTP upload. Supported CSV types: claims_history, customer_profiles, agent_performance, policy_data.

- Validate CSV schema (required columns, data types)
- Clean data (handle nulls, normalize strings, parse dates)
- Transform into internal Pydantic schemas
- Load into SQLite or PostgreSQL via SQLAlchemy
- Return summary: records processed, errors, skipped

6.5 Folder Structure

```
(root)/
  main.py           # FastAPI app: router registration, lifespan, CORS
  config.py         # Pydantic BaseSettings – env-driven configuration
  requirements.txt  # All Python dependencies
  ml/
    train_model.py  # Model training script → outputs fraud_model.pkl
    predictor.py    # Model loading and inference helpers
  models/
    database.py     # SQLAlchemy engine, session factory, create_tables()
    schemas.py      # Pydantic request/response schemas
```

```

routers/
  fraud_router.py      # /predict/fraud/* endpoints
  etl_router.py        # /etl/* endpoints
  analytics_router.py  # /analytics/* endpoints
services/
  fraud_detection.py   # RuleBasedDetector + MLFraudDetector classes
  etl_service.py       # CSV ingestion and data cleaning logic
  analytics_service.py # Aggregation queries for metrics
data/sample/          # Sample CSVs (claims, customers, agents, policies)
scripts/
  generate_data.py     # Generate synthetic training data
  seed_db.py           # Populate DB from sample CSVs
tests/                # Pytest test suite
```

6.6 Quick Start

```

git checkout dev-python-ml-service
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python scripts/seed_db.py # optional: seed sample data
uvicorn main:app --reload --port 8000
# API Docs: http://localhost:8000/docs

# To train and enable the ML model:
python scripts/generate_data.py
python ml/train_model.py # outputs ml/fraud_model.pkl
# Then set FRAUD_DETECTION_MODE=ML in .env
```

7. Service 4 — Node.js Notification Service

Node.js 18 +
Socket.IO +
Express

Branch: dev-node-notifications-backend | Port: 5001 | MySQL

The Notification Service is an independently deployable microservice that acts as the real-time communication backbone of InsurancelQ. It is triggered by Spring Boot whenever a significant business event occurs and delivers push notifications to connected React clients over WebSocket. It also runs a daily scheduled cron job for policy renewal reminders.

7.1 WebSocket Architecture

Connection Setup

The React client connects with a JWT in the Socket.IO handshake. The socketManager middleware validates the token and joins each socket to two rooms: a personal room (user_<userId>) and a role broadcast room (role_<ROLE>).

```
// React client connection
const socket = io('http://localhost:5001', {
  auth: { token: localStorage.getItem('token') }
});

// Server room assignment on connect
socket.join(`user_${userId}`); // personal
socket.join(`role_${role}`);   // broadcast to all admins, etc.
```

Notification Payload Shape

```
{
  "id": 123,
  "type": "CLAIM_APPROVED",
  "title": "Your claim has been approved",
  "message": "Claim #CLM-2024-001 approved for settlement.",
  "read": false,
  "createdAt": "2024-11-15T10:30:00Z"
}
```

7.2 Cron Jobs

Job	Schedule / Description
-----	------------------------

Policy Renewal Reminder	Runs daily at 00:00. Queries Spring Boot for policies expiring in the next 30 days. Pushes renewal reminder notifications to affected customers and their assigned agents.
-------------------------	--

7.3 REST Endpoints

Method + Endpoint	Auth	Description
GET /health	Public	Service health check
POST /events/trigger	JWT required	Spring Boot triggers a notification event
GET /notifications	JWT required	Fetch all notifications for current user
PATCH /notifications/:id/read	JWT required	Mark a specific notification as read
PATCH /notifications/read-all	JWT required	Mark all notifications as read
DELETE /notifications/:id	JWT required	Delete a notification

7.4 Database Schema

```
-- Notifications database: insuranceiq_notifications

CREATE TABLE notifications (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  type        VARCHAR(100) NOT NULL,
  title        VARCHAR(255) NOT NULL,
  message      TEXT NOT NULL,
  is_read      BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE events (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  event_type   VARCHAR(100) NOT NULL,
  payload      JSON,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

7.5 Quick Start

```
git checkout dev-node-notifications-backend
cd notification-service && npm install
cp .env.example .env # set DB credentials + JWT_SECRET
mysql -u root -p insuranceiq_notifications < schema.sql
```

```
npm run dev          # http://localhost:5001
```

8. Service 5 — Cloud & DevOps Deployment

AWS EKS +
Docker +
Jenkins +
Prometheus

Branch: dev-cloud-deployment | AWS Region: us-east-1

The cloud deployment layer containerizes all four application services and orchestrates them on AWS EKS. It uses Jenkins for automated CI/CD, AWS ECR for image storage, AWS EBS for MySQL persistence, and a Prometheus/Grafana stack for monitoring. NGINX Ingress provides path-based routing via a single AWS Network Load Balancer.

8.1 Dockerfiles

Service	Base Image	Build Type	Exposed Port
React Frontend	node:20-alpine → nginx:alpine	Multi-stage	80
Spring Boot Backend	maven:3.9.6-temurin-17 → jre-alpine	Multi-stage	8080
Python ML Service	python:3.11-slim	Single-stage	8000
Node.js Notification	node:20-alpine	Single-stage	5001

Key notes on each Dockerfile:

- React: Accepts VITE_SPRING_API_URL and VITE_SOCKET_URL as Docker build-time ARGS so values are baked in at build time
- Spring Boot: All DB credentials and service URLs are injected as environment variables at container runtime — nothing hardcoded in the image
- Python: Installs gcc for native binary dependencies (pymysql); creates data/sample and data/uploads directories
- Node.js: Uses npm ci --only=production to exclude devDependencies from the production image

8.2 Kubernetes Resource Map

Resource Name	Kind	Namespace	Description
(cluster namespace)	Namespace	insuranceiq	Dedicated namespace isolating all platform resources
frontend-deployment	Deployment	insuranceiq	React Nginx, 2 replicas, port 80
backend-deployment	Deployment	insuranceiq	Spring Boot, 2 replicas, port 8080

ml-service-deployment	Deployment	insuranceiq	Python FastAPI, 1 replica, port 8000
notification-deployment	Deployment	insuranceiq	Node.js Socket.IO, 1 replica, port 5001
mysql-statefulset	StatefulSet	insuranceiq	MySQL 8.0 with EBS PersistentVolumeClaim
mysql-pvc	PersistentVolumeClaim	insuranceiq	10Gi gp2 EBS volume for MySQL data
insuranceiq-ingress	Ingress	insuranceiq	NGINX: /, /api, /ml, /socket.io routing
insuranceiq-secrets	Secret	insuranceiq	DB passwords, JWT secret (base64 encoded)
insuranceiq-configmap	ConfigMap	insuranceiq	Service URLs, DB config, non-sensitive settings

8.3 NGINX Ingress Routing

Path Prefix	Backend Service : Port
/	frontend-service : 80
/api	backend-service : 8080
/ml	ml-service : 8000
/socket.io	notification-service : 5001

8.4 CI/CD Pipeline (Jenkins)

The Jenkins pipeline uses Groovy Jenkinsfile with the following stages:

- Checkout — Pull latest source from GitHub
- Build — mvn clean package (Spring Boot), npm install (React, Node.js), pip install (Python)
- Test — mvn test, pytest, npm test (Jest/Supertest), npm test (Vitest/RTL) — all must pass
- Docker Build — Build all 4 images tagged with ECR repository URI
- ECR Push — Authenticate to AWS ECR via aws ecr get-login-password, push all images
- Deploy — kubectl set image to rolling-update each Deployment in the EKS cluster

8.5 Kubernetes Secrets & ConfigMap

Secret: insuranceiq-secrets

```
stringData:
```

```
SPRING_DATASOURCE_PASSWORD: <db-password>
APP_JWT_SECRET: <256-bit-jwt-secret>
DB_PASSWORD: <notification-db-password>
JWT_SECRET: <same-as-APP_JWT_SECRET>
```

ConfigMap: insuranceiq-configmap

```
data:
  SPRING_DATASOURCE_URL: "jdbc:mysql://mysql-service:3306/insuranceiq_db..."
  APP_FRAUD_SERVICE_URL: "http://ml-service:8000"
  APP_NOTIFICATION_SERVICE_URL: "http://notification-service:5001"
  DB_HOST: "mysql-service"
  DB_NAME: "insuranceiq_notifications"
  SPRING_BOOT_URL: "http://backend-service:8080"
```

8.6 Monitoring — Prometheus & Grafana

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
# Default login: admin / prom-operator
```

Pre-built Grafana dashboards cover: Kubernetes cluster health (CPU, memory, pod restarts), HTTP request rates and latencies per service, MySQL connection pool and query stats, and Node.js event loop lag and active socket connections.

8.7 Deployment Walkthrough

Step 1 — Provision the EKS Cluster:

```
eksctl create cluster --name insuranceiq-cluster --region us-east-1 \
  --nodegroup-name workers --node-type t3.medium --nodes 3
```

Step 2 — Create ECR Repositories:

```
aws ecr create-repository --repository-name insuranceiq-frontend
aws ecr create-repository --repository-name insuranceiq-backend
aws ecr create-repository --repository-name insuranceiq-ml-service
aws ecr create-repository --repository-name insuranceiq-notification
```

Step 3 — Build, Tag and Push Images:

```
aws ecr get-login-password --region us-east-1 | \
```

```
docker login --username AWS --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com
docker build -t insuranceiq-backend ./server
docker tag insuranceiq-backend:latest <account>.dkr.ecr.us-east-1.amazonaws.com/insuranceiq-backend:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/insuranceiq-backend:latest
# Repeat for frontend, ml-service, and notification service
```

Step 4 — Deploy all Kubernetes manifests:

```
kubectl apply -f k8s/
```

Step 5 — Get the public endpoint:

```
kubectl get ingress -n insuranceiq # copy the NLB DNS address
```

9. Data Model & Database Schema

9.1 Core Database — insuranceiq_db

Owned and managed by the Spring Boot service. Uses MySQL 8.0 with Spring Data JPA / Hibernate (ddl-auto=update). All tables are auto-created on first startup.

Table	Key Columns	Notes
users	id, name, email, password_hash, role, status	All platform users (admin, agent, customer, claims_manager)
agents	agent_id, user_id, name, license_no, region, commission_pct, policies_sold	FK: user_id → users
customers	customer_id, user_id, name, phone, dob, address, kyc_status, agent_id	FK: user_id → users, agent_id → agents
insurance_products	product_id, name, type, coverage_amount, premium_rate, term_months	Catalog: MOTOR, HEALTH, PROPERTY, LIFE
policies	policy_id, customer_id, agent_id, product_id, start/end_date, premium, status, fraud_risk_score	FK: customer, agent, product; status: ACTIVE/EXPIRED/CANCELLED/LAPSED
claims	claim_id, policy_id, customer_id, claim_type, incident_date, claim_amount, status, fraud_score	status: PENDING/APPROVED/REJECTED/SETTLED
claim_documents	doc_id, claim_id, doc_type, s3_url, uploaded_at	FK: claim_id → claims
payments	payment_id, policy_id, amount, payment_date, type, status	type: PREMIUM or SETTLEMENT
fraud_predictions	prediction_id, claim_id, fraud_probability, risk_status, recommendation	FK: claim_id → claims

9.2 Notification Database — insuranceiq_notifications

Owned and managed by the Node.js notification service. Uses a separate MySQL database with two tables.

Table	Description
notifications	id, user_id, type, title, message, is_read, created_at — stores all user notifications
events	id, event_type, payload (JSON), created_at — audit log of all triggered events

9.3 ML Database — insuranceiq.db (SQLite) or MySQL (Prod)

Owned and managed by the Python ML service. Uses SQLite by default for local development. Switching to MySQL or PostgreSQL requires setting the DATABASE_URL environment variable. Tables are auto-created on startup via SQLAlchemy metadata.

10. Authentication & Security

10.1 JWT Configuration

Property	Value / Notes
Algorithm	HS256 — HMAC with SHA-256
Token expiry	24 hours (86,400,000 ms) — configurable
Secret env var (Spring Boot)	APP_JWT_SECRET
Secret env var (Node.js)	JWT_SECRET — must be identical to Spring Boot secret
Token claims	userId, role, sub (email)
Transport	Authorization: Bearer <token> HTTP header
Storage (React)	localStorage — cleared on 401 response
WebSocket auth	socket.handshake.auth.token — validated by socketManager middleware

10.2 Roles & Permissions

Role	Dashboard	Management Access	Restricted Pages
admin	AdminDashboard	All customers, policies, claims, agents	Full platform access
agent	AgentDashboard	Own customers, own policies	Claims Workflow, Fraud Report, Analytics
customer	CustomerDashboard	Own policies and claims only	All admin/agent pages
claims_manager	No dedicated dash	Claims Workflow, Fraud Reports, Analytics	Customer/Policy management

11. Local Development Setup

To run the full platform locally, start all four services in separate terminals. All services must be running for full functionality.

11.1 Prerequisites

Tool	Required Version / Notes
Java JDK	17 or higher
Apache Maven	3.8 or higher
Node.js	18.x or higher
npm	9.x or higher
Python	3.11 or higher
pip / venv	Bundled with Python 3.11+
MySQL	8.0 running locally or via Docker

11.2 Setup Steps

Step 1 — Create MySQL Databases

```
mysql -u root -p
CREATE DATABASE insuranceiq_db;
CREATE DATABASE insuranceiq_notifications;
exit;
```

Step 2 — Start Spring Boot (Terminal 1)

```
git checkout dev-spring-backend
cd server
# Edit application.properties with your MySQL credentials
mvn spring-boot:run    # http://localhost:8080
```

Step 3 — Start Python ML Service (Terminal 2)

```
git checkout dev-python-ml-service
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && cp .env.example .env
python scripts/seed_db.py
uvicorn main:app --reload --port 8000    # http://localhost:8000
```

Step 4 — Start Node.js Notification Service (Terminal 3)

```
git checkout dev-node-notifications-backend
```

```
cd notification-service && npm install
cp .env.example .env
mysql -u root -p insuranceiq_notifications < schema.sql
npm run dev    # http://localhost:5001
```

Step 5 — Start React Frontend (Terminal 4)

```
git checkout dev-react-frontend
cd client && npm install
echo "VITE_SPRING_API_URL=http://localhost:8080/api" > .env
echo "VITE_SOCKET_URL=http://localhost:5001" >> .env
npm run dev    # http://localhost:5173
```

11.3 Service Health Check URLs

Service	URL
React Frontend	http://localhost:5173
Spring Boot API (Swagger)	http://localhost:8080/swagger-ui.html
Python ML Service (Docs)	http://localhost:8000/docs
Node.js Notification Service	http://localhost:5001/health

12. Environment Variables Reference

12.1 React Frontend (client/.env)

Variable	Default / Description
VITE_SPRING_API_URL	http://localhost:8080/api — Spring Boot REST base URL
VITE_SOCKET_URL	http://localhost:3001 — Node.js WebSocket server URL

12.2 Spring Boot (application.properties / env)

Variable	Default / Description
SPRING_DATASOURCE_URL	jdbc:mysql://localhost:3306/insuranceiq_db?... — MySQL connection string
SPRING_DATASOURCE_USERNAME	root — MySQL username
SPRING_DATASOURCE_PASSWORD	root — MySQL password (change in production)
APP_JWT_SECRET	Long default key — HS256 secret. MUST change in production
APP_FRAUD_SERVICE_URL	http://localhost:8000 — Python ML service base URL
APP_NOTIFICATION_SERVICE_URL	http://localhost:5001 — Node.js notification service URL
APP_CORS_ORIGINS	http://localhost:5173,3000,80 — comma-separated allowed origins

12.3 Python ML Service (.env)

Variable	Default / Description
DATABASE_URL	sqlite:///./insuranceiq.db — SQLAlchemy DB URL (use postgres:// in prod)
FRAUD_DETECTION_MODE	RULE_BASED — set to ML after training the model
ML_MODEL_PATH	ml/fraud_model.pkl — path to the trained scikit-learn model
HOST	0.0.0.0 — server bind address
PORT	8000 — server port
CORS_ORIGINS	localhost:3000,8080,5173 — comma-separated allowed origins
DEBUG	True — enables auto-reload and verbose logging

12.4 Node.js Notification Service (notification-service/.env)

Variable	Default / Description
PORT	5001 — HTTP and Socket.IO server port
DB_HOST	localhost — MySQL host
DB_PORT	3306 — MySQL port
DB_NAME	insuranceiq_notifications — notification database name
DB_USER	root — MySQL username
DB_PASSWORD	(required) — MySQL password
JWT_SECRET	(required) — MUST exactly match APP_JWT_SECRET in Spring Boot
SPRING_BOOT_URL	http://localhost:8080 — Spring Boot URL used by cron job

Appendix: AWS Services Reference

AWS Service	Used For	Config	Notes
EKS	Managed Kubernetes cluster	t3.medium nodes, 3 replicas min	eksctl create cluster
ECR	Docker image registry	One repo per service (4 total)	Authenticated via aws ecr get-login-password
EBS CSI Driver	MySQL persistent storage	gp2 StorageClass, 10Gi PVC	Installed as EKS addon
NLB	Public ingress entry point	Auto-provisioned by NGINX Ingress	Provides external DNS hostname
IAM	Access control for EKS nodes	Roles for ECR push, EBS driver	Attached to node group IAM role
VPC	Network isolation	Created by eksctl automatically	Private subnets for worker nodes